

PRACTICAL SESSION-4

TASK-1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
vennela_07@VennelaPC:~/PRACTICAL4/TASK1$ nano task1.c
vennela_07@VennelaPC:~/PRACTICAL4/TASK1$ PRACTICAL4/TASK1
-bash: PRACTICAL4/TASK1: No such file or directory
vennela_07@VennelaPC:~/PRACTICAL4/TASK1$ gcc task1.c -o task1
vennela_07@VennelaPC:~/PRACTICAL4/TASK1$ ./task1
Child 1: PID = 566
Parent: Waiting using wait()
Child 2: PID = 567
Child 2 completed
Parent: One child completed using wait()
Child 1 completed
Parent: Child 1 completed using waitpid()
```

```
vennela_07@VennelaPC: ~/PF × + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t child1, child2;
    int status;

    child1 = fork();

    if (child1 == 0) {
        printf("Child 1: PID = %d\n", getpid());
        sleep(2);
        printf("Child 1 completed\n");
        exit(10);
    }

    child2 = fork();

    if (child2 == 0) {
        printf("Child 2: PID = %d\n", getpid());
        sleep(1);
        printf("Child 2 completed\n");
        exit(20);
    }

    printf("Parent: Waiting using wait()\n");

    wait(&status);
    printf("Parent: One child completed using wait()\n");

    waitpid(child1, &status, 0);
    printf("Parent: Child 1 completed using waitpid()\n");

    return 0;
}
```

TASK-2:create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
vennela_07@VennelaPC:~$ cd PRACTICAL4
vennela_07@VennelaPC:~/PRACTICAL4$ pwd
/home/vennela_07/PRACTICAL4
vennela_07@VennelaPC:~/PRACTICAL4$ mkdir TASK2
vennela_07@VennelaPC:~/PRACTICAL4$ cd TASK2
vennela_07@VennelaPC:~/PRACTICAL4/TASK2$ pwd
/home/vennela_07/PRACTICAL4/TASK2
vennela_07@VennelaPC:~/PRACTICAL4/TASK2$ nano zombie.c
vennela_07@VennelaPC:~/PRACTICAL4/TASK2$ gcc zombie.c -o zombie
vennela_07@VennelaPC:~/PRACTICAL4/TASK2$ ./zombie
Parent process: PID = 1395
Parent is sleeping...
Child process: PID = 1396
Child is exiting...
```

```
GNU nano 7.2 zombie.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child is exiting...\n");
        exit(0);
    }
    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Parent is sleeping...\n");
        sleep(30);
    }

    return 0;
}
```

```
vennela_07@VennelaPC:~/PRACTICAL4/TASK2$ nano zombie.c
vennela_07@VennelaPC:~/PRACTICAL4/TASK2$ gcc zombie.c -o zombie
vennela_07@VennelaPC:~/PRACTICAL4/TASK2$ ./zombie
Parent process: PID = 21007
Parent is sleeping...
Child process: PID = 21008
Child is exiting...
^C
vennela_07@VennelaPC:~/PRACTICAL4/TASK2$ ps -o pid,ppid,state,cmd -p 964
  PID   PPID  S  CMD
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        printf("Child process exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent waiting for child...\n");
        printf("Child PID: %d\n", pid);

        wait(&status);

        if (WIFEXITED(status))
        {
            printf("Child completed successfully.\n");
            printf("Child exit status: %d\n", WEXITSTATUS(status));
        }
    }
}
```